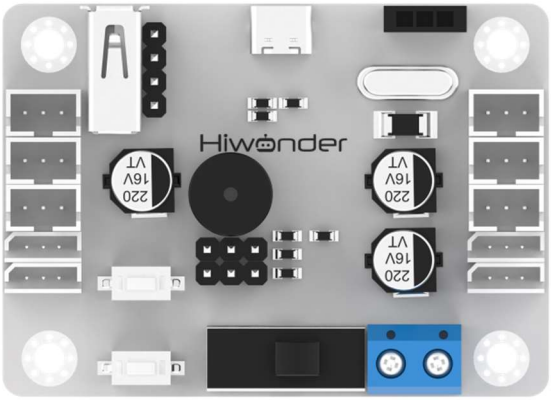
Jetson Development



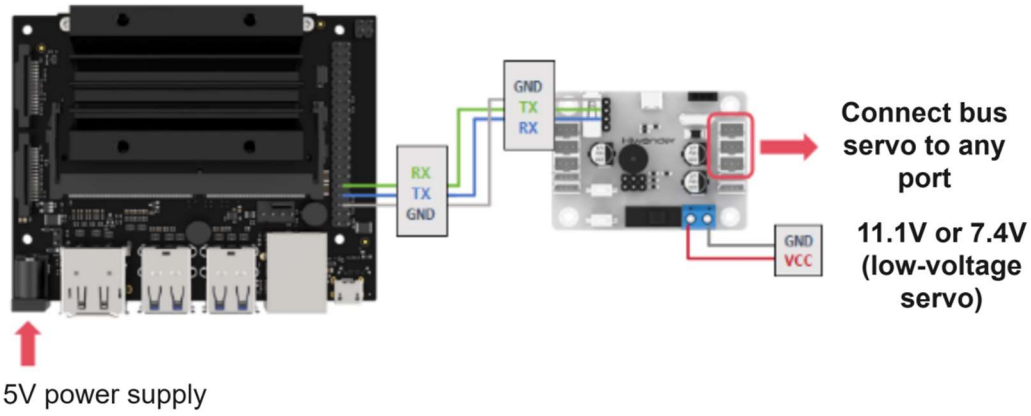
Catalogue

1. Getting Started.....3
1.1 Wiring Instruction.....3
1.2 Environment Configuration.....3
2. Development Case.....4
2.1 Case 1 Control a Single Servo.....4
2.1.1 Run Program.....4
2.1.2 Project Outcome.....5
2.1.3 Program Brief Analysis.....5
2.2 Case 2 Control a Single Servo's Speed.....8
2.2.1 Run Program.....8
2.2.2 Project Outcome.....8
2.2.3 Program Brief Analysis.....8
2.3 Case 3 Control Multiple Servos.....9
2.3.1 Run Program.....9
2.3.2 Project Outcome.....10
2.3.3 Program Brief Analysis.....10
2.4 Case 4 Central Position & Deviation Adjustment.....13
2.4.1 Run Program.....13
2.4.2 Project Outcome.....13
2.4.3 Program Brief Analysis.....13

1. Getting Started

1.1 Wiring Instruction

This example uses an Jetson Nano controller and a bus servo controller, powered by a 11.1V 6000mAh lithium battery, or an 7.4V battery for low-voltage servos. Connect the serial port of the bus servo controller to the serial port of the open-source servo controller.



Note:

① When using Hiwonder's lithium battery, connect the battery cable with the red wire to the positive (+) terminal and the black wire to the negative (–) terminal of the DC port.

② Before connecting the battery cables, make sure they are not already attached to the lithium battery. This prevents the risk of a short circuit caused by accidental contact between the positive and negative wires.

③ When connecting the serial ports, make sure to cross the RX and TX pins.

1.2 Environment Configuration

Install NoMachine software on PC. The software package is stored in "**2. Software -> Remote Connection Software**". For the detailed operations of NoMachine, please refer to the relevant tutorials.

Drag the program and the SDK library files into the Raspberry Pi system

image. This example places the file in the /home/hiwonder/Desktop/Jetson_BusServoControl_demo directory.

Note: Make sure the library files are placed in the same directory as the program.

Open the command-line terminal and enter the following command to add execution permissions:

```
sudo find /home/hiwonder/Desktop/Jetson_BusServoControl_demo -type f -name "*.py" -exec chmod a+x {} \;
```

```
❯ sudo find /home/hiwonder/Desktop/Jetson_BusServoControl_demo -type f -name "*.py" -exec chmod a+x {} \;
```

Note: Replace the directory in the command with the root directory of your own example folder.

2. Development Case

2.1 Case 1 Control a Single Servo

In this example, the serial port is used to send instructions to the bus servo controller, which then controls the servo to move.

2.1.1 Run Program

(1) Open a new terminal and enter the following command to navigate to the program directory:

```
cd /home/hiwonder/Desktop/Jetson_BusServoControl_demo/single_servo_control_turn
```

```
❯ cd /home/hiwonder/Desktop/Jetson_BusServoControl_demo/single_servo_control_turn
~/De/Jetson_BusServoControl_demo/single_servo_control_turn ✔ 12:26:02
```

(2) Run the program by entering:

```
sudo python3 BusServoControl.py
```

```
❯ sudo python3 BusServoControl.py
```

2.1.2 Project Outcome

After running the program, the servo will continuously swing back and forth between position 0 and position 1000, with a 2-second interval.

2.1.3 Program Brief Analysis

- **Import Necessary Libraries**

```
from ServoControl import setBusServoMove
import time
```

The library includes the necessary modules for communicating with the bus servo controller. You can use the predefined variables and functions in it to control the servo. In the `ServoControl` library file, the main method called is `setBusServoMove()`. The time parameter is used to control the delay.

```
#Control single bus servo movement (控制单个总线舵机转动)
def setBusServoMove(servo_id, servo_pulse, time):
    buf = bytearray(b'\x55\x55')  # Frame header (帧头)
    buf.append(0x08) #Data length (数据长度)
    buf.append(LOBOT_CMD_SERVO_MOVE) #Command (指令)
    buf.append(0x01) #Number of servos to control (要控制的舵机个数)

    time = 0 if time < 0 else time
    time = 30000 if time > 30000 else time
    time_list = list(time.to_bytes(2, 'little'))    #Time (时间)
    buf.append(time_list[0])
    buf.append(time_list[1])

    servo_id = 254 if (servo_id < 1 or servo_id > 254) else servo_id
    buf.append(servo_id) #id

    servo_pulse = 0 if servo_pulse < 0 else servo_pulse
    servo_pulse = 1000 if servo_pulse > 1000 else servo_pulse
    pulse_list = list(servo_pulse.to_bytes(2, 'little'))    #
    buf.append(pulse_list[0])
    buf.append(pulse_list[1])

    serialHandle.write(buf)

#Control single PWM servo movement (控制单个PWM舵机转动)
def setPWMServoMove(servo_id, servo_pulse, time):
    buf = bytearray(b'\x55\x55')  # Frame header (帧头)
    buf.append(0x08) #Data length (数据长度)
    buf.append(LOBOT_CMD_SERVO_MOVE) #Command (指令)
    buf.append(0x01) #Number of servos to control (要控制的舵机个数)

    time = 0 if time < 0 else time
    time = 30000 if time > 30000 else time
    time_list = list(time.to_bytes(2, 'little'))    #Time (时间)
    buf.append(time_list[0])
    buf.append(time_list[1])

    servo_id = 254 if (servo_id < 1 or servo_id > 254) else servo_id
    buf.append(servo_id) #舵机ID

    servo_pulse = 500 if servo_pulse < 500 else servo_pulse
    servo_pulse = 2500 if servo_pulse > 2500 else servo_pulse
    pulse_list = list(servo_pulse.to_bytes(2, 'little'))    #Position (位置)
    buf.append(pulse_list[0])
    buf.append(pulse_list[1])

    serialHandle.write(buf)
```

According to the communication protocol, the frame header, data length, command, and the number of servos to be controlled are first sent to the serial port.

```
buf = bytearray(b'\x55\x55')  # Frame header (帧头)
buf.append(0x08) #Data length (数据长度)
buf.append(LOBOT_CMD_SERVO_MOVE) #Command (指令)
buf.append(0x01) #Number of servos to control (要控制的舵机个数)
```

Next, the time parameter passed in is filtered to ensure it falls within the range of (0, 30000). Values outside this range are clamped to 0 and 30000, respectively. This ensures that after the data is sent over the serial port, the controller can correctly recognize it. Then, the time parameter is split into high and low bytes using `list()`, where `'little'` indicates low byte first. These two bytes are then sent to the serial port.

```
time = 0 if time < 0 else time
time = 30000 if time > 30000 else time
time_list = list(time.to_bytes(2, 'little'))    #Time (时间)
buf.append(time_list[0])
buf.append(time_list[1])
```

After that, the ID and position values are also filtered. If the incoming ID is less than 1 or greater than 254, it is clamped to 1 or 254, respectively. Note that 254 means "control all servos." The position value is handled similarly: values below 0 are set to 0.

```
servo_id = 254 if (servo_id < 1 or servo_id > 254) else servo_id
buf.append(servo_id) #id

servo_pulse = 0 if servo_pulse < 0 else servo_pulse
servo_pulse = 1000 if servo_pulse > 1000 else servo_pulse
pulse_list = list(servo_pulse.to_bytes(2, 'little'))    #
buf.append(pulse_list[0])
buf.append(pulse_list[1])
```

Finally, the buf data is sent to the serial port using the write() method.

```
serialHandle.write(buf)
```

- **UART Initialization**

```
serialHandle = serial.Serial("/dev/ttyTHS1", 9600)    # Initialize serial port, baud rate 9600
(初始化串口，波特率为9600)
```

Create an instance of the servo control object and set the baud rate to 9600.

- **Control Servo Movement**

```
if __name__ == '__main__':
    while True:
        setBusServoMove(1, 0, 1000)
        time.sleep(2)
        setBusServoMove(1, 1000, 1000)
        time.sleep(2)
        time.sleep(2)
```

After the main program runs in `BusServoControl`, it calls the `setBusServoMove()` function to control Servo ID 1 to move from position 0 to 1000.

2.2 Case 2 Control a Single Servo's Speed

In this example, the serial port is used to send instructions to the bus servo controller, which then controls the servo to move.

2.2.1 Run Program

(1) Open a new terminal and enter the following command to navigate to the program directory:

```
cd /home/hiwonder/Desktop/Jetson_BusServoControl_demo/single_servo
_control_speed
```

(2) Run the program by entering:

```
sudo python3 BusServoSpeed.py
```

2.2.2 Project Outcome

After running the program, the servo will rotate from position 0 to position 1000 at a duration set to 1200 ms, and then rotate back from position 1000 to position 0 at a duration set to 2000 ms.

2.2.3 Program Brief Analysis

- **Import Necessary Libraries**

```
from ServoControl import setBusServoMove
import time
```

The library includes the necessary modules for communicating with the bus servo controller. You can use the predefined variables and functions in it to control the servo. In the `ServoControl` library, the main method used is `setBusServoMove()`, which changes the servo speed by adjusting the

movement duration. The `time` parameter is used to control the delay.

- **UART Initialization**

```
serialHandle = serial.Serial("/dev/ttyTHS1", 9600)    # Initialize serial port, baud rate 9600
(初始化串口，波特率为9600)
```

Create an instance of the servo control object and set the baud rate to 9600.

- **Control Servo Movement**

```
if __name__ == '__main__':
    while True:
        setBusServoMove(1, 0, 1200)
        time.sleep(2)
        setBusServoMove(1, 1000, 1200)
        time.sleep(2)
        setBusServoMove(1, 0, 2000)
        time.sleep(4)
        setBusServoMove(1, 1000, 2000)
        time.sleep(4)
```

In the main program of `BusServoControl`, the `setBusServoMove()` function is first called to move Servo ID 1 from position 0 to 1000 over 1200 ms. Then, Servo ID 1 is moved back from position 1000 to 0 over 2000 ms. When the rotation angle is the same, a shorter duration results in a higher speed.

2.3 Case 3 Control Multiple Servos

In this example, the serial port is used to send instructions to the bus servo controller, which then controls multiple servos to move.

2.3.1 Run Program

(1) Open a new terminal and enter the following command to navigate to the program directory:

```
cd /home/hiwonder/Desktop/Jetson_BusServoControl_demo/multi_servo_control
```

```
cd /home/hiwonder/Desktop/Jetson_BusServoControl_demo/multi_servo_control
~/De/Jetson_BusServoControl_demo/multi_servo_control  14:23:08
```

(2) Run the program by entering:

```
sudo python3 BusServoMoveByArray.py
```

```
sudo python3 BusServoMoveByArray.py
```

2.3.2 Project Outcome

After running the program, Servo 1 and Servo 2 will move back and forth between position 0 and position 1000, with an interval of 1.5 seconds. The number of servos you can control depends on the available ports on the controller. In this example, only two servos are used, but you can modify the code to control more servos.

2.3.3 Program Brief Analysis

- **Import Necessary Libraries**

```
from ServoControl import setBusServoMove
import time
```

The library includes the necessary modules for communicating with the bus servo controller. You can use the predefined variables and functions in it to control the servo. In this example, the `setBusServoMoveByArray()` method is mainly used to control multiple servos. The `time` parameter is used to control the delay.

```
#Control single bus servo movement (控制单个总线舵机转动)
def setBusServoMove(servo_id, servo_pulse, time):
    buf = bytearray(b'\x55\x55')  # Frame header (帧头)
    buf.append(0x08) #Data length (数据长度)
    buf.append(LOBOT_CMD_SERVO_MOVE) #Command (指令)
    buf.append(0x01) #Number of servos to control (要控制的舵机个数)

    time = 0 if time < 0 else time
    time = 30000 if time > 30000 else time
    time_list = list(time.to_bytes(2, 'little'))    #Time (时间)
    buf.append(time_list[0])
    buf.append(time_list[1])

    servo_id = 254 if (servo_id < 1 or servo_id > 254) else servo_id
    buf.append(servo_id) #id

    servo_pulse = 0 if servo_pulse < 0 else servo_pulse
    servo_pulse = 1000 if servo_pulse > 1000 else servo_pulse
    pulse_list = list(servo_pulse.to_bytes(2, 'little'))    #
    buf.append(pulse_list[0])
    buf.append(pulse_list[1])

    serialHandle.write(buf)

#Control single PWM servo movement (控制单个PWM舵机转动)
def setPWMServoMove(servo_id, servo_pulse, time):
    buf = bytearray(b'\x55\x55')  # Frame header (帧头)
    buf.append(0x08) #Data length (数据长度)
    buf.append(LOBOT_CMD_SERVO_MOVE) #Command (指令)
    buf.append(0x01) #Number of servos to control (要控制的舵机个数)

    time = 0 if time < 0 else time
    time = 30000 if time > 30000 else time
    time_list = list(time.to_bytes(2, 'little'))    #Time (时间)
    buf.append(time_list[0])
    buf.append(time_list[1])

    servo_id = 254 if (servo_id < 1 or servo_id > 254) else servo_id
    buf.append(servo_id) #舵机ID

    servo_pulse = 500 if servo_pulse < 500 else servo_pulse
    servo_pulse = 2500 if servo_pulse > 2500 else servo_pulse
    pulse_list = list(servo_pulse.to_bytes(2, 'little'))    #Position (位置)
    buf.append(pulse_list[0])
    buf.append(pulse_list[1])

    serialHandle.write(buf)
```

According to the communication protocol, the frame header, data length, command, and the number of servos to be controlled are first sent to the serial port.

```
buf = bytearray(b'\x55\x55')  # Frame header (帧头)
buf.append(0x08) #Data length (数据长度)
buf.append(LOBOT_CMD_SERVO_MOVE) #Command (指令)
buf.append(0x01) #Number of servos to control (要控制的舵机个数)
```

Exception filtering for servo count and rotation time is performed to ensure that the bus servo controller can correctly recognize the data sent via the serial port.

```
servos_count = 1 if servos_count < 1 else servos_count
servos_count = 254 if servos_count > 254 else servos_count
buf.append(servos_count) #Number of servos to control (要控制的舵机个数)

time = 0 if time < 0 else time
time = 30000 if time > 30000 else time
time_list = list(time.to_bytes(2, 'little'))
buf.append(time_list[0])    #Time (时间)
buf.append(time_list[1])
```

A `for` loop is used to send data from the `servos` list to the serial port. Set every two elements as one group: the first element is the servo ID, and the second is the target position. For example: `list = [1, 500, 2, 300]` means that servo ID 1 will move to position 500, and servo ID 2 will move to position 300. Finally, the `buf` data is sent to the serial port using the `write()` method.

```
for i in range(servos_count):
    buf.append(servos[i*2]) #Servo ID (舵机ID)

    pos = servos[i*2+1]
    pos = 500 if pos < 500 else pos
    pos = 2500 if pos > 2500 else pos
    pos_list = list(pos.to_bytes(2, 'little'))
    buf.append(pos_list[0])    #Position (位置)
    buf.append(pos_list[1])

serialHandle.write(buf)
```

- **UART Initialization**

```
serialHandle = serial.Serial("/dev/ttyTHS1", 9600)    # 初始化串口， 波特率为9600
```

Create an instance of the servo control object and set the baud rate to 9600.

- **Control Servo Movement**

```
if __name__ == '__main__':
    while True:
        servos = [1, 1000, 4, 300]
        setMoreBusServoMove(servos, 2, 1000)
        time.sleep(2)
        servos = [1, 0, 4, 700]
        setMoreBusServoMove(servos, 2, 1000)
        time.sleep(2)
```

In the main program of `ServoControl`, a `servos` queue is first created to set servo ID 1 to position 1000 and servo ID 4 to position 300. Then, the `setBusServoMoveByArray()` function is called to move these two servos to their target positions within 1000 ms. After a 2-second delay, servo ID 1 is set to position 0 and servo ID 2 to position 700. Then, the `setBusServoMoveByArray()` function is called again to move the servos, also with a duration of 1000 ms.

2.4 Case 4 Central Position & Deviation Adjustment

In this example, the serial port is used to send instructions to the bus servo controller, which then controls the servo to move to central position and adjusts its deviation.

The central position is considered the initial position of the servo and serves as the zero point for rotating in both positive and negative directions. Therefore, the servo should be adjusted to its central position before attaching the servo horn.

Servo deviation is caused by the spacing of the splines on the servo arm—a mechanical limitation that can be corrected through software. In robotics, if this deviation is not addressed, it may restrict the movement of the robot and negatively impact certain motion behaviors.

2.4.1 Run Program

(1) Open a new terminal and enter the following command to navigate to the program directory:

```
cd /home/hiwonder/Desktop/Jetson_BusServoControl_demo/servo_adjust
```

(2) Run the program by entering:

```
sudo python3 BusServoMedAndBias.py
```

2.4.2 Project Outcome

The servo first returns to the central position. After a short delay, it rotates to the specified deviation angle and holds that position.

2.4.3 Program Brief Analysis

- **Import Necessary Libraries**

```
from ServoControl import setBusServoMove
import time
```

The library includes the necessary modules for communicating with the bus servo controller. You can use the predefined variables and functions in it to control the servo. In the `ServoControl` library file, the main method called is `setBusServoMove()`. The `time` parameter is used to control the delay.

- **UART Initialization**

```
serialHandle = serial.Serial("/dev/ttyTHS1", 9600)    # 初始化串口， 波特率为9600
```

Create an instance of the servo control object and set the baud rate to 9600.

- **Control Servo Movement**

```
if __name__ == '__main__':
    deviation = 30
    setBusServoMove(1, 500, 800)
    time.sleep(2)
    setBusServoMove(1, 500+deviation, 800)
    while True:
        time.sleep(1)
```

In the main program of `BusServoControl`, a variable named `deviation` is first created to store the deviation value. The `setBusServoMove()` function is initially called to move servo ID 1 to position 500 within 800 ms. After a 2-second delay, the servo is moved again based on its previous position (500) plus the `deviation` value, thereby achieving a software-based deviation adjustment.